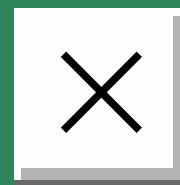
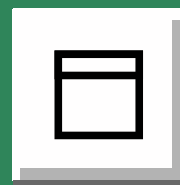
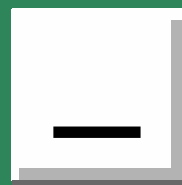
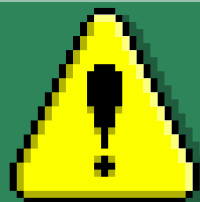


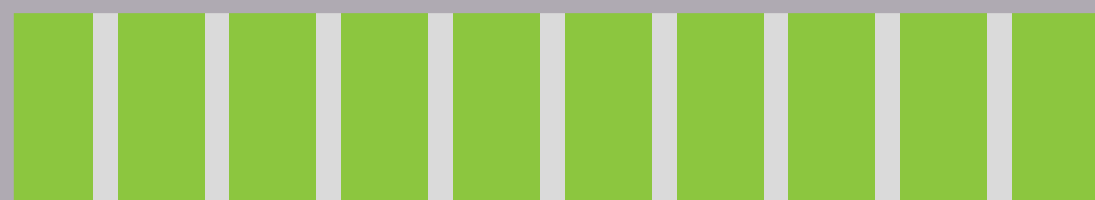


Daskom
Laboratory.



POINTER

Stop



MODUL 9

#Looping
Forever



Tujuan Pembelajaran

1. Memahami konsep pointer dan array.
2. Menguasai operasi pointer.
3. Mampu mendefinisikan dan menginisialisasi pointer.
4. Menggunakan pointer dalam program.
5. Memahami hubungan pointer dan array, pointer dan string, serta pointer dan struct.
6. Menerapkan konsep pointer dan array, pointer dan string, serta pointer dan struct.

#Looping
Forever



Pengertian pointer

- Sebuah tipe data yang menyimpan alamat dimana suatu data disimpan
- Menggunakan tanda '&' untuk mengakses alamat
- Dan tanda '*' untuk mengakses nilai
- Sintaks : `TipeData *NamaPointer;`

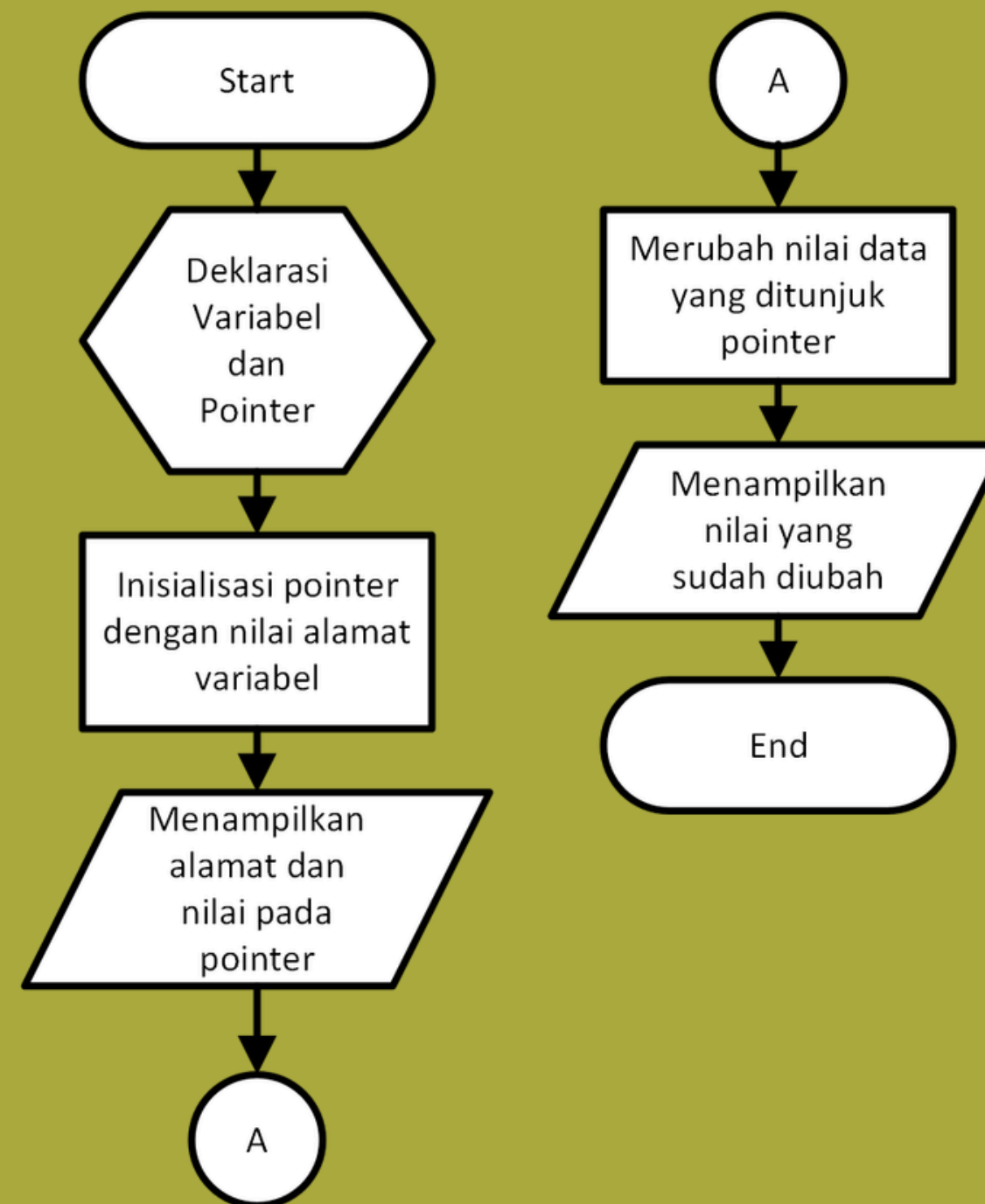


Kegunaan Pointer

- Memungkinkan modifikasi data langsung pada memori
- Alokasi memori dinamis
- Mengubah nilai variabel diluar fungsi tanpa mengembalikan nilai hasil

#Looping
Forever

Alur Penggunaan Pointer



#Looping
Forever

Sintaks Pointer



Daskom
Laboratory.

```
1  #include<stdio.h>
2
3  int main(){
4      int *intptr;
5      char *charptr;
6      float *floatptr;
7      void *voidptr;
8      struct StructName *structptr;
9
10     int value = 550;
11     int *valueptr = &value; // Deklarasi dan Inisialisasi pointer
12     printf("Alamat : %p\n", valueptr); // Output alamat
13     printf("Nilai : %d\n", *valueptr); // Output nilai
14     *valueptr = 707; // Merubah nilai
15     printf("Nilai Baru : %d", *valueptr); // Output nilai baru
16
17     return 0;
18 }
```



```
1  Alamat : 0000005e653ff6c4
2  Nilai  : 550
3  Nilai Baru : 707
```

#Looping
Forever



Pointer Void

- **Dapat menyimpan alamat dari variabel dengan tipe data manapun**
- **Tidak memiliki informasi terkait tipe data alamat yang tersimpan**
- **Menggunakan Explicit Typcasting**

**#Looping
Forever**



Explicit Typcasting

- Konversi tipe data suatu variabel
- Dilakukan saat mengakses nilai variabel untuk memberitahu jenis tipe data yang diakses
- Sintaks : (TipeData)Variabel

Sintaks Pointer Void



Daskom
Laboratory.

```
1  #include<stdio.h>
2
3  int main(){
4      int *intptr;
5      char *charptr;
6      float *floatptr;
7      void *voidptr;
8      struct StructName *structptr;
9
10     int value = 550;
11     int *valueptr = &value; // Deklarasi dan Inisialisasi pointer
12     printf("Alamat : %p\n", valueptr); // Output alamat
13     printf("Nilai : %d\n", *valueptr); // Output nilai
14     *valueptr = 707; // Merubah nilai
15     printf("Nilai Baru : %d", *valueptr); // Output nilai baru
16
17     return 0;
18 }
```



```
1  Alamat : 0000005e653ff6c4
2  Nilai  : 550
3  Nilai Baru : 707
```

#Looping
Forever



Pointer, Array & String

Cara kerja pointer mirip dengan array, ini dikarenakan array merupakan pointer konstan yang menyimpan alamat memori data.

String pun juga merupakan array yang terdiri dari char sehingga memiliki prinsip yang sama juga

Oleh karena itu :

- Elemen array dan String dapat diakses menggunakan notasi pointer
- Pointer dapat digunakan untuk membuat array dinamis

#Looping
Forever

Akses Array Dengan Notasi Pointer



Daskom
Laboratory.



```
1  #include <stdio.h>
2
3  int main() {
4      int array[] = {9, 18, 27, 36, 45};
5      int *pointer_array = array;
6
7      printf("Elemen ke-1 %d\n", *pointer_array);
8      pointer_array++; // alamat memori dimajukan satu kali
9      printf("Elemen ke-2 %d\n", *pointer_array);
10     pointer_array += 2; // alamat memori dimajukan dua kali
11     printf("Elemen ke-4 %d\n", *pointer_array);
12     return 0;
13 }
```



```
1  Elemen ke-1 9
2  Elemen ke-2 18
3  Elemen ke-4 36
```

#Looping
Forever

Akses String Dengan Notasi Pointer



Daskom
Laboratory.



```
1  #include <stdio.h>
2
3  int main() {
4      char *string = "Program Bahasa C";
5
6      printf("String awal : %s\n", string);
7      string++; // alamat memori digeser satu kali
8      printf("Setelah digeser sebanyak 1 kali : %s\n", string);
9      string += 2; // alamat memori digeser dua kali
10     printf("Setelah digeser lagi sebanyak 2 kali : %s\n", string);
11     return 0;
12 }
```



```
1  String awal : Program Bahasa C
2  Setelah digeser sebanyak 1 kali : rogram Bahasa C
3  Setelah digeser lagi sebanyak 2 kali : gram Bahasa C
```

#Looping
Forever



Alokasi Memori

Dengan menggunakan library `<stdlib.h>` kita dapat mengalokasikan memori secara dinamis dengan menggunakan fungsi - fungsi berikut :

- `malloc()`; Mengalokasi blok memori
- `realloc()`; Mengubah ukuran alokasi memori
- `calloc()`; Mengalokasi sejumlah blok memori
- `free()`; Melepas memori yang sudah dialokasikan

#Looping
Forever



Pembuatan Array Dinamis

Berbeda dengan array biasa yang statis, ukuran array dinamis dapat berubah - ubah selama program berjalan. Untuk menghasilkan ini kita dapat menggunakan dua fungsi alokasi, yaitu :

- **malloc();** sebagai inisialisasi dan alokasi ukuran array awal
- **realloc();** mengubah ukuran array yang telah dialokasi

#Looping
Forever



Daskom
Laboratory.



```
1 Array Awal
2 Elemen 0 : 1
3 Elemen 1 : 3
4 Elemen 2 : 6
5
6 Array Berukuran Baru
7 Elemen 0 : 1
8 Elemen 1 : 3
9 Elemen 2 : 6
10 Elemen 3 : 12
11 Elemen 4 : 24
```



Pembuatan Array Dinamis



```
1 #include <stdio.h>
2 #include <stdlib.h> // Agar bisa menggunakan fungsi alokasi
3
4 int main() {
5     int *arr = malloc(3 * sizeof(int)); // Alokasi awal array
6     arr[0] = 1; // Memasukkan elemen ke array awal
7     arr[1] = 3;
8     arr[2] = 6;
9     printf("Array Awal\n");
10    for(int i = 0; i < 3; i++){
11        printf("Elemen %d : %d\n", i, arr[i]);
12    }
13    arr = realloc(arr, 5 * sizeof(int)); // Realokasi memori baru, dimana ukuran array membesar
14    arr[3] = 12; // Memasukan elemen baru yang sekarang muat di dalam array
15    arr[4] = 24;
16    printf("\nArray Berukuran Baru\n");
17    for(int i = 0; i < 5; i++){
18        printf("Elemen %d : %d\n", i, arr[i]);
19    }
20    return 0;
21 }
```

#Looping
Forever



Pointer dan Struct

- **Berguna untuk mengelompokkan sejumlah data dengan tipe data bervariasi**
- **Dengan pointer, ukurannya dapat diperkecil karena hanya pointer hanya menyimpan alamat**

Pembuatan Pointer Struct

```
1  #include<stdio.h>
2  #include<stdlib.h> // stdlib.h untuk menggunakan malloc()
3  // Deklarasi Struct
4  struct Mahasiswa{
5      char nama[80];
6      int nim;
7  };
8
9  int main(){
10     struct Mahasiswa *mhs; // Deklarasi dan alokasi data struct
11     mhs = (struct Mahasiswa*)malloc(sizeof(struct Mahasiswa));
12     if(mhs == NULL){printf("Alokasi memori gagal\n"); return 1;}
13     // Menerima input langsung ke pointer struct
14     printf("Input nama   : ");gets(mhs->nama);
15     printf("Masukkan NIM : ");scanf("%d", &mhs->nim);
16     // Mengeluarkan output langsung dari pointer struct
17     printf("Nama Mahasiswa : %s\n", mhs->nama);
18     printf("NIM  Mahasiswa : %d\n", mhs->nim);
19
20     return 0;
21 }
```

#Looping
Forever

```
1  Input nama   : Alden
2  Masukkan NIM : 101012
3  Nama Mahasiswa : Alden
4  NIM  Mahasiswa : 101012
```




Pointer dan Fungsi

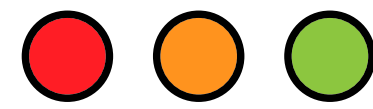
- **Pointer bisa digunakan sebagai parameter sebuah fungsi**
- **Dengan pointer, sebuah fungsi void dapat memodifikasi nilai suatu variabel diluar fungsi tersebut**
- **Sehingga mengembalikan sebuah nilai tidak diperlukan**



Daskom
Laboratory.



```
1 Angka Semula      : 9
2 Angka Pangkat 2    : 81
```



Pointer dalam Fungsi



```
1  #include<stdio.h>
2  void Pangkat2(int *nilai); // Pastikan parameter berupa pointer
3
4  int main(){
5      int angka = 9;
6
7      printf("Angka Semula      : %d\n", angka);
8      Pangkat2(&angka); // Masukkan argumen berupa pointer
9      printf("Angka Pangkat 2 : %d", angka);
10
11     return 0;
12 }
13
14 void Pangkat2(int *nilai){
15     *nilai = *nilai * *nilai; // Merubah data langsung pada alamatnya
16     return;
17 }
```

#Looping
Forever

